

**Master MSDE 7**

*École Hassania des Travaux Publics*

**Module 5 — Machine Learning**

# Détection de Transactions Bancaires Frauduleuses

*Mise en œuvre d'un pipeline ML complet  
et déploiement opérationnel via Streamlit*

Réalisé par :

**FANANE SARA**

**MOUNIR MARIA**

*Année académique 2025/2026*

## Sommaire

1. Introduction
2. AXE 1 — Compréhension de la problématique
3. AXE 2 — Analyse exploratoire des données
4. AXE 3 — Pré-processing des données
5. AXE 4 — Construction et évaluation des modèles ML
6. AXE 5 — Tuning des modèles et choix du modèle final
7. AXE 6 — Sérialisation du pipeline complet
8. AXE 7 — Déploiement de l'application Streamlit
9. Conclusion et perspectives
10. Références bibliographiques

## 1. Introduction

La fraude par carte de crédit constitue aujourd'hui l'un des défis majeurs auxquels font face les institutions financières mondiales. Selon le Nilson Report 2023, les pertes mondiales liées à la fraude par carte ont atteint 34 milliards de dollars en 2022, et continuent de croître avec l'accélération de la digitalisation des paiements.

Dans ce contexte, le Machine Learning offre une opportunité unique d'automatiser la détection de transactions frauduleuses en temps réel, à partir de l'analyse statistique de millions de transactions historiques. Ce projet, réalisé dans le cadre du Module 5 « Machine Learning » vise à mettre en pratique un pipeline ML complet, depuis la compréhension du problème métier jusqu'au déploiement opérationnel d'une application web de prédiction.

Le présent rapport décrit la démarche méthodologique adoptée, les choix techniques effectués, ainsi que les résultats obtenus à chaque étape du projet. Il est structuré selon les sept axes définis dans l'énoncé du projet.

## 2. AXE 1 — Compréhension de la problématique

### 2.1 Contexte métier

Pour une banque, chaque fraude non détectée représente une perte financière directe (remboursement obligatoire au client), une dégradation de la confiance des porteurs de cartes, et un risque réglementaire. À l'inverse, chaque fausse alerte (transaction légitime bloquée à tort) génère une friction client et un coût opérationnel important. Le défi est donc double : détecter un maximum de fraudes tout en limitant le nombre de fausses alertes.

### 2.2 Description du dataset

Le dataset utilisé provient de Kaggle, publié par le Machine Learning Group de l'Université Libre de Bruxelles (ULB). Il contient des transactions réelles effectuées par carte de crédit en septembre 2013 par des porteurs de cartes européens, sur une période de deux jours.

| Caractéristique        | Valeur                           |
|------------------------|----------------------------------|
| Nombre de transactions | 284 807                          |
| Nombre de features     | 31                               |
| Variable cible         | Class (1 = fraude, 0 = légitime) |
| Nombre de fraudes      | 492                              |
| Proportion de fraudes  | 0.172%                           |
| Période couverte       | 2 jours (septembre 2013)         |

Pour des raisons de confidentialité bancaire, les variables originales ne sont pas révélées. Seules deux variables sont interprétables : Time (secondes écoulées depuis la première transaction) et Amount (montant en euros). Les 28 autres variables (V1 à V28) sont les composantes principales issues d'une transformation PCA appliquée aux variables originales.

### 2.3 Traduction en tâche ML

Le problème métier se traduit en une tâche de classification binaire supervisée :

- Type : Classification binaire supervisée
- Variable cible : Class (0 = légitime, 1 = fraude)
- Features : 30 variables numériques continues
- Challenge principal : Déséquilibre extrême (0.172% de fraudes)
- Challenge secondaire : Anonymisation par PCA, difficulté d'interprétation

### 2.4 État de l'art

La détection de fraude par carte de crédit est un domaine de recherche actif depuis plus de deux décennies. Les approches développées dans la littérature peuvent être classées en plusieurs grandes familles :

#### Approches statistiques et linéaires

Les premières approches reposaient sur des modèles linéaires (régression logistique) ou des règles métier expertes. Ces méthodes restent utilisées comme baseline mais peinent à capturer les patterns complexes de fraude dans des données à haute dimension.

### Méthodes ensemblistes

Les méthodes ensemblistes basées sur le gradient boosting (XGBoost, LightGBM, CatBoost) dominent aujourd'hui l'état de l'art sur les données tabulaires. Dal Pozzolo et al. (2015) ont popularisé l'utilisation de ces méthodes spécifiquement sur le dataset utilisé dans ce projet.

### Détection d'anomalies

Lorsque les fraudes sont très rares ou non labellisées, les approches non supervisées (Isolation Forest, One-Class SVM, Autoencoders) sont utiles pour identifier des fraudes de types inédits.

### Techniques de rééquilibrage

Le déséquilibre des classes est traité par des techniques de rééchantillonnage. SMOTE (Chawla et al., 2002) génère des exemples synthétiques de la classe minoritaire par interpolation entre voisins. C'est la technique retenue dans ce projet.

### Apprentissage profond

Des architectures plus récentes (MLP, LSTM, GNN) ont été explorées, mais sur des datasets tabulaires de taille modérée comme le nôtre, elles n'apportent généralement pas de gain significatif par rapport aux méthodes ensemblistes classiques.

## 3. AXE 2 — Analyse exploratoire des données

### 3.1 Vue d'ensemble

Le dataset compte 284 807 lignes et 31 colonnes. Toutes les variables sont numériques continues. Aucune valeur manquante n'a été détectée, ce qui simplifie considérablement le preprocessing.

### 3.2 Analyse de la variable cible

L'analyse de la variable Class révèle un déséquilibre extrême : sur 284 807 transactions, seulement 492 sont frauduleuses, soit 0.172%. Ce déséquilibre est typique des problèmes de détection d'événements rares (fraude, défaillance, maladie rare) et impose des choix méthodologiques spécifiques :

- L'accuracy ne peut pas être utilisée comme métrique (un modèle naïf prédisant « légitime » obtiendrait 99.83%).
- Le rééquilibrage des classes via SMOTE est nécessaire pour l'entraînement.
- Les métriques PR-AUC et G-Mean sont privilégiées.

### 3.3 Analyse des variables Amount et Time

Amount présente une distribution très asymétrique avec des valeurs allant de 0 à 25 691€, et une médiane de 22€. Cette forte asymétrie justifie l'utilisation du RobustScaler (basé sur la médiane et l'IQR) plutôt que le StandardScaler (basé sur la moyenne et l'écart-type), plus sensible aux valeurs extrêmes.

Time est exprimé en secondes depuis la première transaction et couvre 172 800 secondes (soit 48 heures). On observe deux pics correspondant aux périodes diurnes, séparés par une période creuse nocturne. Time est conservé comme feature après scaling.

### 3.4 Corrélations avec la variable cible

L'analyse de corrélation linéaire entre chaque variable et la cible révèle que les variables V17, V14, V12, V10, V11, V16 et V3 sont les plus négativement corrélées à Class, c'est-à-dire qu'elles tendent à prendre des valeurs plus faibles pour les transactions frauduleuses. À l'inverse, V11, V4 et V2 présentent une corrélation positive.

Ces résultats orientent l'analyse et seront confirmés lors de la Feature Selection en AXE 3.

### 3.5 Conclusions de l'EDA

L'EDA a permis d'identifier les points suivants, qui orientent les choix de preprocessing :

- Pas de valeurs manquantes → aucune imputation nécessaire.
- Variables V1-V28 déjà normalisées par PCA → seules Amount et Time nécessitent un scaling.
- RobustScaler choisi pour gérer les outliers d'Amount sans suppression.
- Déséquilibre extrême → SMOTE pour le rééquilibrage du train set.

## 4. AXE 3 — Pré-processing des données

### 4.1 Étapes appliquées

Le preprocessing suit l'ordre suivant pour garantir l'absence de data leakage :

- Scaling de Amount et Time avec RobustScaler.
- Séparation Train / Test (80/20) avec stratification.
- Feature Selection sur le train uniquement.
- SMOTE appliqué uniquement sur le train.

### 4.2 Traitement des outliers

La variable Amount contient des transactions à valeurs très élevées (jusqu'à 25 691€). Nous avons délibérément choisi de ne pas supprimer ces outliers, pour quatre raisons :

- Le RobustScaler est insensible aux valeurs extrêmes (médiane + IQR).
- Les modèles ensemblistes (Random Forest, XGBoost, Extra Trees) sont robustes aux outliers.
- SMOTE n'amplifie pas les outliers de la classe majoritaire (il agit uniquement sur les fraudes).
- Justification métier : les fraudes peuvent justement avoir des montants élevés.

### 4.3 Train / Test Split

Le split est réalisé avec `test_size=0.2`, `random_state=42` et `stratify=y`. La stratification est essentielle pour préserver le ratio fraudes/légitimes dans les deux ensembles, sans quoi le test set pourrait avoir un nombre insuffisant de fraudes pour une évaluation fiable.

| Ensemble            | Taille  | Légitimes | Fraudes | Ratio fraudes |
|---------------------|---------|-----------|---------|---------------|
| Train (avant SMOTE) | 227 845 | 227 451   | 394     | 0.173%        |
| Test                | 56 962  | 56 864    | 98      | 0.172%        |

### 4.4 Feature Selection

Deux approches complémentaires ont été utilisées pour évaluer la pertinence des features :

- SelectKBest avec ANOVA F-test : mesure statistique de la relation linéaire entre chaque feature et la cible.
- Random Forest Feature Importance : mesure la contribution non-linéaire de chaque feature dans un modèle ensembliste.

Les deux approches convergent : V17, V14, V12, V10 et V11 ressortent comme les variables les plus discriminantes, ce qui confirme les corrélations observées en EDA. Cependant, toutes les features présentent une  $p$ -value  $< 0.05$ , ce qui signifie qu'aucune n'est statistiquement non significative.

Décision finale : conservation de toutes les 30 features, car les modèles ensemblistes effectuent une sélection interne automatique via leur mécanisme de split gain, et les variables PCA sont orthogonales par construction (chacune apporte une information non redondante).

## 4.5 Rééquilibrage par SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) génère des exemples synthétiques de fraudes par interpolation entre les voisins les plus proches. Contrairement à un oversampling naïf qui dupliquerait les exemples (et provoquerait du surapprentissage), SMOTE crée des points nouveaux dans l'espace des features.

Le SMOTE est appliqué exclusivement sur le train set, jamais sur le test. Cela évite tout data leakage et garantit que les performances mesurées sur le test reflètent les conditions réelles d'utilisation. Après SMOTE, le train passe de 227 845 transactions (avec 394 fraudes) à 454 902 transactions parfaitement équilibrées (227 451 légitimes et 227 451 fraudes).



## 5. AXE 4 — Construction et évaluation des modèles ML

### 5.1 Choix des métriques d'évaluation

Compte tenu du déséquilibre extrême, les métriques classiques (Accuracy, ROC-AUC) sont inadaptées. Nous avons retenu deux métriques complémentaires :

#### PR-AUC (Precision-Recall AUC)

Métrique principale, mesure l'aire sous la courbe Precision-Recall. Elle évalue la capacité du modèle à détecter les fraudes (Recall) tout en limitant les fausses alertes (Precision). Contrairement au ROC-AUC, elle se concentre exclusivement sur la classe minoritaire et n'est pas influencée par le grand nombre de transactions légitimes. Saito & Rehmsmeier (2015) ont démontré que la PR-AUC est plus informative que le ROC-AUC sur les datasets très déséquilibrés.

#### G-Mean (Geometric Mean)

Métrique de contrôle, calculée comme la racine carrée du produit de la sensibilité (recall sur la classe positive) et de la spécificité (recall sur la classe négative). Sa nature multiplicative pénalise les modèles qui sacrifient une classe au profit de l'autre.

### 5.2 Algorithmes testés

Onze algorithmes ont été testés, couvrant les principales familles de la littérature en classification supervisée :

| #  | Algorithme          | Famille                     |
|----|---------------------|-----------------------------|
| 1  | Logistic Regression | Linéaire                    |
| 2  | Decision Tree       | Arbre simple                |
| 3  | K-Nearest Neighbors | Instance-based              |
| 4  | Naive Bayes         | Probabiliste                |
| 5  | Random Forest       | Bagging                     |
| 6  | Extra Trees         | Bagging (splits aléatoires) |
| 7  | XGBoost             | Boosting                    |
| 8  | LightGBM            | Boosting (histogrammes)     |
| 9  | CatBoost            | Boosting                    |
| 10 | AdaBoost            | Boosting adaptatif          |
| 11 | Gradient Boosting   | Boosting standard           |

### 5.3 Résultats comparatifs

Tous les modèles ont été entraînés avec leurs hyperparamètres par défaut. Le tableau ci-dessous présente leurs performances classées par PR-AUC décroissante :

| Rang | Modèle              | PR-AUC | Recall | Precision | G-Mean |
|------|---------------------|--------|--------|-----------|--------|
| 1    | Extra Trees         | 0.879  | 0.847  | 0.892     | 0.920  |
| 2    | Random Forest       | 0.868  | 0.816  | 0.879     | 0.903  |
| 3    | XGBoost             | 0.867  | 0.857  | 0.718     | 0.926  |
| 4    | LightGBM            | 0.844  | 0.878  | 0.528     | 0.936  |
| 5    | CatBoost            | 0.825  | 0.847  | 0.509     | 0.920  |
| 6    | AdaBoost            | 0.771  | 0.908  | 0.054     | 0.940  |
| 7    | Logistic Regression | 0.724  | 0.918  | 0.059     | 0.946  |
| 8    | Gradient Boosting   | 0.711  | 0.908  | 0.109     | 0.947  |
| 9    | KNN                 | 0.610  | 0.878  | 0.430     | 0.936  |
| 10   | Decision Tree       | 0.286  | 0.765  | 0.373     | 0.874  |
| 11   | Naive Bayes         | 0.084  | 0.878  | 0.054     | 0.924  |

## 5.4 Analyse des résultats

Les modèles ensemblistes (Extra Trees, Random Forest, XGBoost, LightGBM, CatBoost) dominent largement le classement avec une PR-AUC supérieure à 0.82, ce qui confirme l'état de l'art sur ce type de problème.

Naive Bayes et Decision Tree affichent les performances les plus faibles : Naive Bayes souffre de son hypothèse d'indépendance entre features (peu réaliste après PCA), et un arbre de décision seul a tendance à surapprendre le bruit introduit par SMOTE.

AdaBoost et Logistic Regression présentent un Recall élevé mais une Précision très faible (5-6%), signe d'un nombre excessif de fausses alertes : ces modèles ne sont pas exploitables en production.

Les 3 meilleurs modèles (Extra Trees, Random Forest, XGBoost) sont retenus pour l'étape de tuning.

## 6. AXE 5 — Tuning des modèles et choix du modèle final

### 6.1 Méthodologie de tuning

Le tuning des hyperparamètres a été réalisé via `RandomizedSearchCV` plutôt que `GridSearchCV`, pour deux raisons :

- Efficacité computationnelle : sur 454 902 lignes (post-SMOTE), une grille exhaustive serait prohibitive.
- Efficacité statistique : la recherche aléatoire permet d'explorer une grande diversité de configurations avec un nombre limité d'essais.

Configuration commune aux trois tunings :

- `n_iter` = 30 combinaisons par modèle.
- `cv` = 3 folds en validation croisée stratifiée.
- `scoring` = `average_precision` (équivalent PR-AUC en sklearn).
- `n_jobs` = -1 pour exploiter tous les cœurs CPU.

### 6.2 Résultats du tuning

Le tableau ci-dessous présente les performances avant et après tuning pour chacun des trois candidats :

| Modèle        | PR-AUC avant | PR-AUC après | Gain absolu | Gain relatif |
|---------------|--------------|--------------|-------------|--------------|
| Extra Trees   | 0.8788       | 0.8799       | +0.0011     | +0.13%       |
| Random Forest | 0.8680       | 0.8776       | +0.0096     | +1.11%       |
| XGBoost       | 0.8673       | 0.8694       | +0.0021     | +0.24%       |

### 6.3 Analyse des gains

Random Forest bénéficie le plus du tuning, avec un gain de +1.11% sur la PR-AUC. Cela suggère que ses paramètres par défaut sklearn n'étaient pas optimaux pour ce dataset. Extra Trees et XGBoost connaissent des gains plus marginaux, ce qui est cohérent avec leur nature : Extra Trees utilise déjà des splits aléatoires (rendant le tuning des autres paramètres moins critique), et XGBoost est connu pour avoir des paramètres par défaut déjà bien calibrés.

### 6.4 Choix du modèle final

Le modèle final retenu est Extra Trees tuné, pour les raisons suivantes :

- Meilleure PR-AUC après tuning (0.8799).
- Meilleure Précision (0.8925) — limite le mieux les fausses alertes.
- G-Mean élevé (0.9202) — bon équilibre des deux classes.
- Temps d'entraînement raisonnable (~15 secondes).

Hyperparamètres optimaux retenus : `n_estimators=300`, `max_depth=None`, `min_samples_split=2`, `min_samples_leaf=1`, `max_features='sqrt'`.

## 6.5 Vérification de l'overfitting

L'analyse des performances train vs test révèle un écart d'environ 0.12 sur la PR-AUC (1.000 sur le train vs 0.8806 sur le test). Cet écart est caractéristique des forêts d'arbres profonds non régularisées : sans limite de profondeur, le modèle mémorise parfaitement le train tout en généralisant correctement aux données non vues.

Ce comportement n'est pas un overfitting pathologique, mais une propriété inhérente aux Extra Trees. Les performances sur le test restent excellentes (Recall 84.69%, Precision 90.22% sur un dataset à 0.172% de fraudes), démontrant une bonne capacité de généralisation.

## 6.6 Performance opérationnelle finale

Le modèle Extra Trees tuné évalué sur le test set (jamais vu lors de l'entraînement) atteint les résultats suivants :

| Métrique              | Valeur      |
|-----------------------|-------------|
| PR-AUC                | 0.8806      |
| Recall (fraudes)      | 84.69%      |
| Precision (fraudes)   | 90.22%      |
| F1-Score              | 0.8737      |
| G-Mean                | 0.9202      |
| Fraudes détectées     | 83 / 98     |
| Fraudes manquées (FN) | 15          |
| Fausse alertes (FP)   | 10 / 56 864 |

## 7. AXE 6 — Sérialisation du pipeline complet

### 7.1 Construction du pipeline

Conformément à la consigne du projet, l'ensemble du preprocessing a été intégré dans un Pipeline scikit-learn, garantissant que l'application Streamlit puisse recevoir des données brutes sans reproduire manuellement les étapes de transformation.

Le pipeline contient deux étapes principales :

- Préprocesseur (ColumnTransformer) : applique le RobustScaler sur les colonnes Amount et Time, et laisse passer les variables V1-V28 (déjà normalisées par PCA).
- Modèle Extra Trees : avec ses hyperparamètres optimaux.

### 7.2 Sérialisation et optimisation

Le pipeline initial sérialisé avec joblib occupait environ 158 MB, principalement à cause de la taille du modèle Extra Trees (300 arbres profonds). Cette taille pose problème pour le déploiement :

- GitHub limite les fichiers à 100 MB par défaut.
- Streamlit Cloud charge le modèle en mémoire au démarrage.

Solution adoptée : compression gzip de niveau maximal via le paramètre `compress=9` de `joblib.dump()`. Cette compression réduit la taille du fichier de 75% sans modifier le modèle ni ses performances. Le fichier final passe ainsi à environ 40 MB, parfaitement compatible avec les contraintes de déploiement.

### 7.3 Validation

Le pipeline sérialisé a été rechargé et testé sur cinq exemples (trois transactions légitimes et deux fraudes). Tous les exemples ont été correctement classifiés avec une excellente séparation des probabilités : les transactions légitimes obtiennent une probabilité de fraude inférieure à 3%, tandis que les fraudes obtiennent une probabilité supérieure à 99%. Cela confirme la validité du pipeline sérialisé.

## 8. AXE 7 — Déploiement de l'application Streamlit

### 8.1 Architecture de l'application

L'application Streamlit propose une interface web simple et ergonomique pour la prédiction en temps réel. Elle se compose des éléments suivants :

- En-tête : titre de l'application et métriques du modèle (PR-AUC, Recall, Precision, F1-Score).
- Sidebar : saisie des features de la transaction (Time, Amount, V1 à V28), avec trois modes prédéfinis : Saisie manuelle, Exemple Légitime, Exemple Fraude.
- Zone principale : affichage des données saisies (colonne gauche) et du résultat de la prédiction (colonne droite).
- Résultat : message clair (TRANSACTION LÉGITIME ou FRAUDE DÉTECTÉE), niveau de confiance affiché en pourcentage, barres de progression visuelles.
- Section À propos : documentation du projet, méthodologie, performances et auteurs.

### 8.2 Fonctionnement

L'application charge le pipeline sérialisé au démarrage (via `@st.cache_resource` pour éviter les rechargements répétés). Lorsque l'utilisateur clique sur « Analyser la transaction », l'application :

- Construit un DataFrame pandas avec les valeurs saisies, dans l'ordre exact attendu par le pipeline.
- Appelle `pipeline.predict()` et `pipeline.predict_proba()` pour obtenir la classe prédite et les probabilités.
- Affiche le résultat avec un code couleur (vert pour légitime, rouge pour fraude) et le niveau de confiance.

### 8.3 Déploiement sur Streamlit Cloud

L'application est déployée gratuitement sur Streamlit Cloud, plateforme officielle de déploiement pour les applications Streamlit. Les fichiers nécessaires au déploiement sont :

- `app.py` : code source de l'application.
- `requirements.txt` : liste des dépendances Python (streamlit, pandas, scikit-learn, xgboost, joblib, etc.).
- `fraud_detection_pipeline.pkl` : pipeline sérialisé compressé.

L'application est accessible publiquement via une URL générée par Streamlit Cloud, qui peut être partagée librement.

## 9. Conclusion et perspectives

### 9.1 Synthèse

Ce projet a permis de mettre en œuvre un pipeline ML complet pour la détection de transactions bancaires frauduleuses, depuis la compréhension du problème métier jusqu'au déploiement opérationnel. La démarche méthodologique adoptée a abouti à un modèle Extra Trees tuné atteignant une PR-AUC de 0.8806 sur le test set, avec un Recall de 84.69% et une Précision de 90.22%.

Concrètement, sur les 56 962 transactions du test set comprenant 98 fraudes, le modèle a correctement détecté 83 fraudes tout en ne générant que 10 fausses alertes sur 56 864 transactions légitimes (taux de fausses alertes : 0.018%). Ces performances sont exploitables en contexte opérationnel.

### 9.2 Apprentissages clés

- L'importance du choix des métriques : sur des datasets déséquilibrés, l'Accuracy et le ROC-AUC sont trompeurs. La PR-AUC s'impose comme la métrique de référence.
- Le rôle critique du rééquilibrage : SMOTE a permis d'améliorer significativement la détection, mais doit être strictement limité au train pour éviter le data leakage.
- La domination des méthodes ensemblistes : sur les données tabulaires déséquilibrées, Random Forest, Extra Trees et XGBoost surpassent largement les modèles linéaires.
- Le compromis recherche/production : le modèle scientifique optimal n'est pas toujours adapté aux contraintes de déploiement. Une optimisation complémentaire (compression du fichier sérialisé) a permis de respecter les limites tout en préservant les performances.

### 9.3 Perspectives d'amélioration

- Stacking ou voting des trois meilleurs modèles tunés pour potentiellement améliorer encore les performances.
- Ajustement du seuil de décision selon le coût métier des FN vs FP (par défaut 0.5, mais peut être abaissé pour augmenter le Recall).
- Exploration d'approches temporelles (LSTM, GRU) si des données comportementales par utilisateur étaient disponibles.
- Combinaison avec un détecteur d'anomalie non supervisé (Isolation Forest, Autoencoder) pour identifier des fraudes de types inédits.
- Monitoring en production avec détection de la dérive du modèle (concept drift) et réentraînement périodique.

## 10. Références bibliographiques

- [1] Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2015). Credit card fraud detection: A realistic modeling and a novel learning strategy. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8), 3784-3797.
- [2] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
- [3] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794).
- [4] Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3).
- [5] Bahnsen, A. C., Aouada, D., Stojanovic, A., & Ottersten, B. (2016). Feature engineering strategies for credit card fraud detection. *Expert Systems with Applications*, 51, 134-142.
- [6] Nilson Report (2023). Card fraud worldwide. Issue 1232.
- [7] Kaggle Credit Card Fraud Detection dataset. URL: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>